Modèle Mathématique pour un Ordonnancement Dynamique des Réplicas Kubernetes basé sur un Budget Énergétique Une

approche formelle pour le GreenOps

MENRA W. ROMIAL

4 août 2025

Résumé

Ce document présente un modèle mathématique et algorithmique détaillé pour ajuster dynamiquement le nombre de réplicas des déploiements dans un cluster Kubernetes en réponse à des variations de budget énergétique ou de puissance électrique. Face à la volatilité des prix de l'énergie et à la nécessité croissante d'une informatique durable, ce modèle propose une solution qui va au-delà d'une simple réduction linéaire. Il intègre la notion de groupes de déploiements, de poids de criticité, de contraintes utilisateur (min/max réplicas) et gère rigoureusement les défis liés aux arrondis et à la répartition juste des efforts de réduction. L'objectif est de fournir un cadre formel, déterministe et équitable pour opérer des infrastructures cloud de manière plus efficiente sur le plan énergétique.

Table des matières

1 Introduction **2**
1.1 Contexte . . . 2
1.2 Problématique . . . 2

2 Formalisation du Problème **2**

3 Modèle Mathématique et Algorithmique **3**
3.1 Étape 1 : Calcul de l'Objectif de Réduction Global . . . 3
3.2 Étape 2 : Répartition de l'Objectif entre Entités . . . 3
3.2.1 Calcul des potentiels maximaux . . . 3
3.2.2 Calcul des objectifs de réduction bruts . . . 3
3.2.3 Attribution des cibles entières : Méthode du Plus Grand Reste . . . 3
3.3 Étape 3 : Calcul de la Réduction Voulue par Déploiement . . . 4
3.3.1 Cas des déploiements indépendants . . . 4
3.3.2 Cas des déploiements groupés . . . 4
3.4 Étape 4 : Ajustement Final sous Contraintes . . . 4
3.4.1 Calcul de la réduction maximale possible . . . 4
3.4.2 Calcul de la réduction initiale et du déficit . . . 4
3.4.3 Redistribution itérative du déficit . . . 5

4 Synthèse du Résultat Final **5**

5 Avantages, Limites et Perspectives **5**
5.1 Avantages du modèle . . . 5
5.2 Limites et Inconvénients Potentiels . . . 5
5.3 Perspectives . . . 6

1 Introduction

1.1 Contexte

L'informatique en nuage (Cloud Computing), et plus particulièrement l'orchestration de conteneurs via des plateformes comme Kubernetes, est devenue la norme pour le déploiement d'applications modernes. Ces systèmes offrent une élasticité sans précédent, permettant aux applications de s'adapter à la charge en faisant varier le nombre d'instances (réplicas). Parallèlement, le monde fait face à une double contrainte : la volatilité des coûts de l'énergie et l'impératif de réduire l'empreinte carbone des infrastructures numériques (.

Dans ce contexte, la consommation électrique d'un cluster Kubernetes n'est plus une métrique secondaire mais un paramètre de premier ordre à optimiser, au même titre que la performance ou le coût.

1.2 Problématique

Le problème fondamental est le suivant : comment aligner la consommation des ressources de calcul, matérialisée par le nombre de réplicas des déploiements, avec un signal externe représentant une contrainte de puissance ou de budget énergétique ?

Une approche naïve consisterait à appliquer une réduction linéaire à tous les déploiements (par exemple, "-20% d'énergie = -20% de réplicas partout"). Cette méthode présente cependant des lacunes majeures :

- **Criticité variable :** Toutes les applications n'ont pas la même importance. Un service de frontend peut être moins critique qu'une base de données ou un service de paiement.
- **Contraintes techniques :** Chaque application possède un nombre minimum de réplicas requis pour assurer sa haute disponibilité et un nombre maximum définissant son échelle-limite.
- **Gestion des groupes :** Les applications sont souvent composées de microservices interdépendants formant des groupes logiques qui devraient être gérés comme une seule entité.

2 Formalisation du Problème

Pour construire notre modèle, nous devons d'abord définir précisément les ensembles, paramètres et variables qui le composent.

Définition 2.1 (Ensembles)**.** *Nous définissons les ensembles suivants :*

- $\mathcal{D}$ *: L'ensemble de tous les déploiements dans le cluster.*
- $\mathcal{G}$ *: L'ensemble de tous les groupes de déploiements.*
- *Pour chaque groupe* $g \in \mathcal{G}$, $\mathcal{D}_g \subset \mathcal{D}$ *est l'ensemble des déploiements qui appartiennent au groupe* g.
- $\mathcal{D}_I = \mathcal{D} \setminus \left(\bigcup_{g \in \mathcal{G}} \mathcal{D}_g\right)$ *: L'ensemble des déploiements indépendants, n'appartenant à aucun groupe.*

Définition 2.2 (Paramètres)**.** *Pour chaque déploiement* $d \in \mathcal{D}$*, les paramètres suivants sont connus :*

- $p \in [0, 1]$ *: Le pourcentage de réduction global à appliquer (ex :* 0.20 *pour 20%).*
- $C_d \in \mathbb{N}^*$ *: Le nombre de réplicas* ***actuels*** *du déploiement* d.
- $N_d \in \mathbb{N}^*$ *: Le nombre* ***minimum*** *de réplicas requis pour le déploiement* d.
- $M_d \in \mathbb{N}^*$ *: Le nombre* ***maximum*** *de réplicas configuré pour le déploiement* d*. Nous considérons que* $N_d \leq C_d \leq M_d$.
- $w_d \in \mathbb{R}^+$ *: Le* ***poids de répartition*** *(ratio) du déploiement* d*. Ce paramètre n'est utilisé que si* d *appartient à un groupe.*

Définition 2.3 (Variable de Décision)**.** *L'objectif du modèle est de déterminer la valeur de la variable suivante pour chaque déploiement :*

— $x_d \in \mathbb{N}_0$ *: Le nombre entier de réplicas à retirer du déploiement d.*

Le nouveau nombre de réplicas sera alors $C'_d = C_d - x_d$*, avec la contrainte fondamentale que* $C'_d \geq N_d$*.*

3 Modèle Mathématique et Algorithmique

Le modèle se décompose en quatre étapes séquentielles.

3.1 Étape 1 : Calcul de l'Objectif de Réduction Global

La première étape consiste à traduire le pourcentage de réduction p en un nombre entier de réplicas à retirer sur l'ensemble du cluster. Cet objectif est basé sur le potentiel d'échelle maximal de tous les déploiements, reflétant l'intention de l'utilisateur.

L'objectif de réduction global, Δ_{total}, est donné par :

$$\Delta_{\text{total}} = \left\lceil p \cdot \sum_{d \in \mathcal{D}} M_d \right\rceil \tag{1}$$

où $\lceil \cdot \rceil$ est la fonction plafond (arrondi à l'entier supérieur). L'utilisation de cette fonction garantit que l'effort de réduction est au moins égal à ce qui est demandé, adoptant une posture agressive en faveur des économies d'énergie.

3.2 Étape 2 : Répartition de l'Objectif entre Entités

L'objectif global Δ_{total} doit ensuite être réparti entre les différentes entités de gestion : les groupes ($g \in \mathcal{G}$) et les déploiements indépendants ($d \in \mathcal{D}_I$). Cette répartition s'effectue au prorata du potentiel maximal de chaque entité, assurant une distribution équitable basée sur l'échelle configurée.

3.2.1 Calcul des potentiels maximaux

— Potentiel maximum d'un groupe g : $M_g = \sum_{d \in \mathcal{D}_g} M_d$

— Potentiel maximum total du cluster : $M_{\text{total}} = \sum_{d \in \mathcal{D}} M_d = \left(\sum_{g \in \mathcal{G}} M_g\right) + \left(\sum_{d \in \mathcal{D}_I} M_d\right)$

3.2.2 Calcul des objectifs de réduction bruts

L'objectif de réduction brut pour chaque entité est calculé proportionnellement à son potentiel :

$$\Delta_g^{\text{brut}} = \Delta_{\text{total}} \cdot \frac{M_g}{M_{\text{total}}}, \quad \forall g \in \mathcal{G} \tag{2}$$

$$\Delta_d^{\text{brut}} = \Delta_{\text{total}} \cdot \frac{M_d}{M_{\text{total}}}, \quad \forall d \in \mathcal{D}_I \tag{3}$$

3.2.3 Attribution des cibles entières : Méthode du Plus Grand Reste

Les valeurs Δ_g^{brut} et Δ_d^{brut} sont des réels. Pour les convertir en cibles entières (Δ_g, Δ_d) dont la somme est exactement Δ_{total}, nous utilisons la méthode du plus grand reste :

1. **Partie entière :** Pour chaque entité e, on calcule sa part entière $P_e = \lfloor \Delta_e^{\text{brut}} \rfloor$.
2. **Reste à distribuer :** On calcule le nombre total d'unités restantes $R = \Delta_{\text{total}} - \sum_e P_e$.

3. **Partie fractionnaire :** Pour chaque entité e, on calcule sa partie fractionnaire $F_e = \Delta_e^{\text{brut}} - P_e$.
4. **Distribution :** On trie les entités par ordre décroissant de F_e. On attribue une unité de réduction supplémentaire (+1) aux R premières entités de la liste.

Le résultat est un ensemble de cibles entières Δ_g et Δ_d qui respectent $\sum \Delta_g + \sum \Delta_d = \Delta_{\text{total}}$.

3.3 Étape 3 : Calcul de la Réduction Voulue par Déploiement

Nous déterminons maintenant le nombre de réplicas à retirer pour chaque déploiement individuel.

3.3.1 Cas des déploiements indépendants

Pour un déploiement $d \in \mathcal{D}_I$, la réduction voulue est simplement sa cible calculée à l'étape précédente :

$$x_d^{\text{voulue}} = \Delta_d \tag{4}$$

3.3.2 Cas des déploiements groupés

Pour un groupe $g \in \mathcal{G}$, nous devons répartir sa cible Δ_g entre ses déploiements membres $d \in \mathcal{D}_g$. Cette répartition se fait au prorata des poids de répartition w_d.

— Somme des poids du groupe g : $W_g = \sum_{d \in \mathcal{D}_g} w_d$

La réduction voulue brute pour chaque $d \in \mathcal{D}_g$ est :

$$x_d^{\text{brut}} = \Delta_g \cdot \frac{w_d}{W_g} \tag{5}$$

Nous appliquons à nouveau la méthode du plus grand reste pour obtenir les valeurs entières x_d^{voulue}, garantissant que $\sum_{d \in \mathcal{D}_g} x_d^{\text{voulue}} = \Delta_g$.

3.4 Étape 4 : Ajustement Final sous Contraintes

C'est l'étape la plus critique, où nous confrontons la réduction idéale x_d^{voulue} avec la réalité des contraintes ‘min_replicas‘.

3.4.1 Calcul de la réduction maximale possible

Pour chaque déploiement d, la réduction ne peut pas faire tomber le nombre de réplicas en dessous de son minimum N_d.

$$x_d^{\text{max_possible}} = C_d - N_d \tag{6}$$

Cette valeur représente le nombre maximum de réplicas que l'on peut retirer sans violer la contrainte.

3.4.2 Calcul de la réduction initiale et du déficit

La réduction réelle initiale, notée $x_d^{(0)}$, est la valeur voulue, plafonnée par ce qui est possible.

$$x_d^{(0)} = \min(x_d^{\text{voulue}}, x_d^{\text{max_possible}}) \tag{7}$$

Si $x_d^{\text{voulue}} > x_d^{\text{max_possible}}$, cela crée un "déficit" de réduction. Le déficit total à reporter est la somme de tous ces déficits individuels.

$$\text{Déficit} = \sum_{d \in \mathcal{D}} (x_d^{\text{voulue}} - x_d^{(0)}) \tag{8}$$

3.4.3 Redistribution itérative du déficit

Ce déficit doit être comblé en demandant un effort supplémentaire aux déploiements qui ont encore une marge de manœuvre. Nous procédons de manière itérative :

Tant que Déficit > 0 :

1. **Identifier les donneurs :** Déterminer l'ensemble des déploiements donneurs $\mathcal{D}_{\text{donneur}} = \{d \in \mathcal{D} \mid x_d < x_d^{\text{max_possible}}\}$.
2. **Condition d'arrêt :** Si $\mathcal{D}_{\text{donneur}}$ est vide, aucun déploiement ne peut plus être réduit. Arrêter la boucle. Le 'Déficit' restant représente une réduction impossible à réaliser.
3. **Sélectionner le meilleur donneur :** Choisir le donneur d^* qui a la plus grande marge de sécurité restante.
$$d^* = \underset{d \in \mathcal{D}_{\text{donneur}}}{\text{argmax}} \, (x_d^{\text{max_possible}} - x_d) \tag{9}$$
Ce critère privilégie le report de la charge sur les déploiements les plus "résilients".
4. **Mettre à jour :**
$$x_{d^*} \leftarrow x_{d^*} + 1$$
$$\text{Déficit} \leftarrow \text{Déficit} - 1$$

Ce processus garantit que l'effort de réduction est maximisé tout en respectant scrupuleusement toutes les contraintes.

4 Synthèse du Résultat Final

Au terme de l'étape 4, nous obtenons le nombre final de réplicas à retirer pour chaque déploiement, noté x_d^{final}. Le nouvel état du cluster est défini par le nouveau nombre de réplicas pour chaque déploiement d :
$$C'_d = C_d - x_d^{\text{final}} \tag{10}$$
Par construction, ce modèle garantit que $\forall d \in \mathcal{D}, C'_d \geq N_d$.

5 Avantages, Limites et Perspectives

5.1 Avantages du modèle

Le modèle présenté offre plusieurs avantages significatifs par rapport à des approches simplistes :

— **Prévisibilité et Équité :** La répartition basée sur les 'max_replicas' et les poids assure un comportement déterministe et aligné sur l'architecture intentionnelle du système.

— **Respect des Contraintes :** La garantie de ne jamais descendre en dessous du 'min_replicas' est fondamentale pour la stabilité des applications.

— **Robustesse Mathématique :** La gestion rigoureuse des arrondis (méthode du plus grand reste) et des déficits assure que l'objectif de réduction est poursuivi de manière optimale et sans ambiguïté.

— **Flexibilité :** Le système de groupes et de poids permet aux opérateurs de définir des politiques de réduction fines et adaptées à la criticité de leurs services.

5.2 Limites et Inconvénients Potentiels

— **Complexité de mise en œuvre :** Ce modèle requiert le développement d'un contrôleur Kubernetes dédié (un "Operator") pour l'implémenter, ce qui est plus complexe qu'une simple règle de Horizontal Pod Autoscaler (HPA).

— **Latence de décision :** Le calcul, bien que déterministe, implique plusieurs étapes. Pour des clusters de très grande taille, la latence de l'algorithme doit être évaluée pour assurer une réactivité suffisante aux signaux énergétiques.
— **Vision macroscopique :** Le modèle se base uniquement sur le nombre de réplicas. Il ne prend pas en compte d'autres formes d'optimisation comme la réduction des ressources allouées à chaque réplica (CPU/mémoire throttling) ou l'état de santé applicatif (taux d'erreur, latence). Une réduction drastique, même si elle respecte 'min_replicas', pourrait dégrader la qualité de service.

5.3 Perspectives

Ce travail ouvre la voie à plusieurs extensions et recherches futures :
— **Intégration de signaux en temps réel :** Coupler le contrôleur à des API de fournisseurs d'énergie (comme RTE en France pour les signaux EcoWatt) pour un pilotage entièrement automatisé.
— **Modèles hybrides :** Combiner ce scaling horizontal (réduction du nombre de réplicas) avec un scaling vertical (ajustement des ressources par pod) pour une granularité d'action encore plus fine.
— **Intégration de métriques de Qualité de Service (QoS) :** Utiliser des métriques comme le temps de réponse ou le taux d'erreur comme des garde-fous, empêchant une réduction si elle dégrade trop sévèrement l'expérience utilisateur, même si elle est mathématiquement possible.

En conclusion, le modèle formalisé dans ce document constitue une base solide et complète pour la conception de systèmes d'ordonnancement Kubernetes conscients de leur consommation énergétique, un enjeu majeur pour l'avenir de l'informatique durable.